# SysMonitor

# System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

# Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

# Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

# Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-SysMonitor-go.

## SysMonitor - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/SysMonitor. Entry point: ./cmd/sysmonitor. Module: arrowhead/ah5-support-go/sysmonitor. See [DEPLOYMENT.md](DEPLOYMENT.md) for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and [UPSTREAM-README.md](UPSTREAM-README.md). The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

## Listening port

Built-in default 127.0.0.1:8475 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8475"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

## Documented behavior and remaining work

| System | Implemented behavior | Remaining work |
|---|---|---|
| SysMonitor | Authorized AMQPS intake, strict observation validation, atomic dedup/history/outbox, subscriptions, history, MQTT QoS1 forwarding | Java MQTTv5 (Go uses 3.1.1), complete live-broker/UI parity, Java history import, broad schema golden tests |

## Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

# Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-SysMonitor-rust.

## SysMonitor (Rust)

Arrowhead 5.2.2 SysMonitor support system. Entry point src/main.rs is built as binary sysmonitor by the shared crate in [../ah5-support-runtime-rust](../ah5-support-runtime-rust) (arrowhead-support-rust).

- examples/sysmonitor.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/SysMonitor, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin sysmonitor -- <config.json>

## Listening port

Default 127.0.0.1:8475 (Arrowhead default, role sysmonitor). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

## Documented behavior and remaining work

| System | Implemented behavior | Remaining work |
|---|---|---|
| SysMonitor | Native mTLS AMQP intake, authorised binding reconciliation, revision subscriptions, SQLite history/outbox, confirmed MQTT output and retention | Full Java DTO/filter parity and wider operational qualification |

## Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

## Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

# Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

# Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-SysMonitor-go/documentation/SysMonitor/SysMonitor-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/SysMonitor/SysMonitor-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

# SysMonitor

## System Design Description

**Abstract**

This document describes the design and implementation of the SysMonitor support system of Arrowhead 5.2.2 (`ah5-SysMonitor-java-spring`).

# Contents

**1 Overview** **3**

**2 Implementation** **4**
2.1 Implementation language and tools . . . . . 4
2.2 Functional properties implementation . . . . . 4
2.3 Non functional properties implementation . . . . . 5
2.4 Deployment . . . . . 6
2.5 Local modifications . . . . . 6
2.6 Verification . . . . . 6

**3 Services** **7**

**4 References** **7**

**5 Revision History** **9**
5.1 Amendments . . . . . 9
5.2 Quality Assurance . . . . . 9

# 1 Overview

This document describes the SysMonitor system implementation, which realizes the SysMonitor SysD [1]. SysMonitor collects the native `sysMonitor` observations of all systems from the AMQP broker, stores their history and republishes them on the MQTT broker (`sysMonitorData`). In Section 2, we describe implementation details of the system.

# 2 Implementation

This implementation realizes the SysMonitor SysD in `ah5-SysMonitor-java-spring` on the Arrowhead Java Spring pattern (as Blacklist and EventHandler). The Java sources, the private copy of the Arrowhead common library (`vendor/`), the tests, the image and the deployment definition are owned by this folder. The previous implementations are preserved in `legacy/` and are excluded from the build.

## 2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (web, security, data JPA, scheduling) with the vendored Arrowhead common library (`common-utils`, `data-transfer-objects`); RabbitMQ AMQP client for the intake and Eclipse Paho MQTT v5 for the output. Maven produces the Spring Boot JAR `arrowhead-sysmonitor-5.2.2.jar` with the entry point `eu.arrowhead.sysmonitor.SysMonitorMain`.
- Storage: the private MySQL schema `ah_sysmonitor` through JPA (`ddl-auto=validate`), reached with the MariaDB driver and `sslMode=verify-full`. The application account has data-manipulation privileges only.

## 2.2 Functional properties implementation

The following components (package `eu.arrowhead.sysmonitor`) implement the system:

| Component | Responsibility |
|---|---|
| `SysMonitorSystemInfo` | System and service models registered by the common `ApplicationInitListener`: `sysMonitorManagement`, `sysMonitorHistory`, `sysMonitorData` (`generic_mqtts`) and `generalManagement`. The common `SysMonitorPublisher` registers and publishes the system's own `sysMonitor` observations. |
| `api.http.HistoryAPI`, `ManagementAPI`, `GeneralManagementAPI`, `HealthAPI` | The HTTP paths of the native 5.2.2 implementation: `/sysmonitor/history/*`, `/sysmonitor/management/*`, `/sysmonitor/mgmt/general/status` (plus `get-log` and `get-config`) and `/health`, port 8475. |
| `service.AmqpIntake` | Durable queue `SysMonitor.observations` on the `arrowhead` topic exchange; validates each observation against its routing key, stores it, then acknowledges (rejects invalid ones, requeues on storage failure). |
| `service.ObservationFormat` | Normalised observation envelope and derived observation identifier (SHA-256 of routing key and canonical JSON), unchanged from the native implementation. |
| `service.MonitoringService` | Orchestrates the authorized `sysMonitor` providers (`generic_amqps`, `CERT_AUTH`) and applies the bindings derived from the subscriptions or the default subscription. |
| `service.MqttOutput` | MQTT v5 output `arrowhead/<cloud>/sysmonitor/data/<type>/<provider>`, QoS 1, from the outbox. |
| `service.SysMonitorJobs` | Scheduled broker (re)connection, orchestration (`ORCHESTRATION_INTERVAL_SECONDS`), outbox (1 s) and retention prune (`SYSMONITOR_PRUNE_INTERVAL_SECONDS`). |
| `jpa.service.MonitorDbService` | Store of subscriptions (revisions, `If-Match`), bindings, outbox, history and audit. |

Every request passes the Spring filter chain: the common `OutsourcedFilter` (identity token verified with Authentication and bound to the client certificate), `CallerProfileFilter` (certificate profile `sy` or `op`), `SysMonitorAccessFilter` (`sysMonitorHistory` operations decided by ConsumerAuthorization, cached 30 s when granted; every other `/sysmonitor` path only for LocalCloudManager or Sysop) and the common management filter for `/sysmonitor/mgmt/` (Sysop and the whitelist LocalCloudManager). Errors use the Arrowhead error body (`errorMessage`, `errorCode`, `exceptionType`, `origin`) with the status codes of the native implementation.

### 2.2.1 Data handled and stored

Private MySQL schema `ah_sysmonitor` (`deployment/database/init.sql`, `src/main/resources/database`):

| Tables | Content |
|---|---|
| `observation, outbox` | History (retention-bounded, `SYSMONITOR_RETENTION_DAYS`) and MQTT output queue. |
| `subscription, amqp_binding, audit` | Subscriptions, broker bindings and audit log. |
| `logs` | Log4j2 JDBC log of the common library (`get-log`). |

The interfaces exchange JSON objects built at run time; their structure is specified in the IDD documents. The history of the former SQLite store is not migrated; it is retention-bounded.

## 2.3 Non functional properties implementation

### 2.3.1 Security

The HTTPS API requires mutual TLS 1.2/1.3 with the PKCS12 identity and trust stores (`client-auth=need`); client certificates are checked against the CRL of the cloud CA (`arrowhead.ssl.crl-file`). The AMQP intake and the MQTT output use the same system certificate (AMQP SASL EXTERNAL), verify the broker host name and check the broker certificate against the same CRL. The common `SecurityProfileGuard` refuses to start with TLS off, declared authentication, `service.authorization.enabled=false` or plain broker connections, unless `arrowhead.security.profile=development` is selected for an isolated test.

### 2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

### 2.3.3 Internal monitoring

Logs go to stdout, a file under `/app/logs` and the `logs` table. The common `SysMonitorPublisher` publishes the system's own `sysMonitor` observations every 30 s; its health is `alive` only with both broker connections.

### 2.3.4 Configuration

`config/application.properties` holds the deployment settings; the environment names `LOCAL_CLOUD`, `ORGANIZATION`, `AMQP_HOST/PORT`, `MQTT_HOST/PORT`, `SYSMONITOR_*`, `ORCHESTRATION_INTERVAL_SECONDS` and `CRL_FILE` remain placeholders with defaults. Secrets (store, identity and database passwords) are read from `/run/secrets` (Spring config tree).

## 2.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-sysmonitor-standalone:5.2.2`. The local `compose.yaml` joins an operator-selected existing cloud network and adds the private `sysmonitor-db` (MySQL 8.4.11 with mandatory TLS on an internal network, schema and data-manipulation-only account created by `deployment/database`). The HTTPS API uses port 8475. The process runs as UID 10001 with a read-only root filesystem; only `/tmp` and `/app/logs` are writable tmpfs. Key and trust stores, the database CA and the CRL are mounted individually and read-only. This packaging is supplied for operator deployment; existing test-cloud deployments are separate deployment consumers and are not changed by it.

To select this system, copy `.env.example` to `.env`, provision its identity/password, PKCS12 key and trust stores, CRL, database TLS material and passwords, configure network and registry, then run `docker compose up -d --build` from this folder. The database volume must never be shared with another system. Stopping this Compose project preserves its volume and does not stop other systems.

ServiceRegistry is the bootstrap endpoint. Authentication and DynamicServiceOrchestration are looked up there; ConsumerAuthorization and the monitored `sysMonitor` providers are orchestrated through DSO and require the cloud's authorization rules. The AMQP broker and the MQTT v5 output broker must be available for operational readiness: SysMonitor reports degraded (HTTP 503 on `/health` and `generalManagement` status) when AMQP intake or MQTT output is disconnected.

### 2.4.1 Ownership and build boundary

The common library is a private vendored source copy (`vendor/PROVENANCE.json`); there is no shared runtime process, data directory or Maven parent other than Spring Boot. Security fixes to the common library must be propagated to every vendored copy. Build with Java 21 and `mvn -B clean verify` (`scripts/verify.sh` also checks the artifact contents), or use the local Docker build; `scripts/verify-image.sh` checks that the image runs read-only and refuses insecure settings.

## 2.5 Local modifications

Notes on this implementation in the workspace:

- Native Java port (2026-10-16); bindings re-applied on every refresh (2026-10-19).
- Converted to the Arrowhead Java Spring pattern with a private MySQL schema (2026-11-08). HTTP paths, status codes, AMQP routing keys, the observation format and the MQTT topics and payloads are unchanged; error bodies now use the Arrowhead error format, `/health` requires the identity token like every other path, and `generalManagement` also offers `get-log` and `get-config`.

## 2.6 Verification

The project's JUnit tests cover the observation format and identity binding, the store (deduplication, outbox, revisions, filters, prune), orchestration and binding selection, the intake acknowledgement rules, the Consumer-Authorization cache, the secure-profile refusal and a route contract test of every previously served path and method behind the deployed filter chain (unauthenticated, wrong certificate and authorised). Opt-in tests run the whole application against a fresh MySQL container built from `deployment/database` (schema validation, data-manipulation-only grants) and an AMQP to store to MQTT round trip against local RabbitMQ and Mosquitto with mutual TLS. A live Go/Rust federation run is not part of this verification.

# 3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

| **Services produced** | **SysD ref** | **SD ref** | **IDD ref** |
|---|---|---|---|
| `sysMonitorManagement` | [1] | [2] | [3] |
| `sysMonitorHistory` | [1] | [4] | [5] |
| `sysMonitorData` | [1] | [6] | [7] |
| `sysMonitor` | [1] | [8] | [9] |
| | | | |
| **Services consumed** | **SysD ref** | **SD ref** | **IDD ref** |
| serviceDiscovery | [10] | [11] | – |
| identity | [12] | [13] | – |
| serviceOrchestration | [14] | [15] | – |
| authorization | [16] | [17] | – |
| sysMonitor (all providers) | AMQP broker | – | – |
| sysMonitorData | MQTT broker | – | – |

# 4 References

[1] J. Delsing, “SysMonitor – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-SysD.pdf.

[2] ——, “sysMonitorManagement – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-SD.pdf.

[3] ——, “sysMonitorManagement HTTP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-HTTP-IDD.pdf.

[4] ——, “sysMonitorHistory – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-SD.pdf.

[5] ——, “sysMonitorHistory HTTP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-HTTP-IDD.pdf.

[6] ——, “sysMonitorData – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-SD.pdf.

[7] ——, “sysMonitorData MQTT/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-MQTT-IDD.pdf.

[8] ——, “sysMonitor – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (SysMonitor),” Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[14] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[15] ——, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[16] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[17] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-SD.pdf.

# 5 Revision History

## 5.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-10-22 | 5.2.2 | Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace. | Jerker Delsing |
| 2 | 2026-10-25 | 5.2.2 | Independent Java build, private helpers and state, per-system deployment and configuration. | Codex (draft) |
| 3 | 2026-10-30 | 5.2.2 | Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated. | Claude Opus 5.5 (draft) |
| 4 | 2026-11-08 | 5.2.2 | Implementation converted to the Arrowhead Java Spring pattern with a private MySQL schema; implementation, security, deployment and verification sections updated. | Claude Opus 5.5 (draft) |

## 5.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.2.2 | |

# 1 SysMonitor

## 1.1 System overview

Monitoring subscriptions, history and durable observation forwarding. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

## 1.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `sysMonitorHistory`, `sysMonitorManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

## 1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- AMQPS intake and MQTT QoS1 outbox forwarding are implemented. MQTT uses Paho 3.1.1 rather than Java MQTTv5. Full live-broker/UI parity and imported Java history have not been certified. The default subscription is none until configured.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

## 1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

# 2 Implementation

Go 1.26 or newer builds the native binary. State is serialized as private JSON documents; updates clone state, validate, persist and then publish. Server timeouts and bounded message sizes constrain resource use. Persistent outboxes are at-least-once and cannot provide distributed exactly-once delivery. Detailed source, configuration and test instructions are in the repository README and verification report.

## 2.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

| Method | Path | Access |
|---|---|---|
| GET | /sysmonitor/management/status | management |
| GET | /sysmonitor/management/providers | management |
| GET | /sysmonitor/management/subscriptions | management |
| PUT | /sysmonitor/management/subscriptions/{providerSystem} | management |
| DELETE | /sysmonitor/management/subscriptions/{providerSystem} | management |
| POST | /sysmonitor/management/reconcile | management |
| GET | /sysmonitor/history/systems | identity |
| GET | /sysmonitor/history/systems/{systemName}/latest | identity |
| GET | /sysmonitor/history/systems/{systemName}/observations | identity |
| GET | /sysmonitor/monitor/echo | certificate |
| GET | /sysmonitor/general/mgmt/get-config | management |
| POST | /sysmonitor/general/mgmt/logs | management |

# 1 SysMonitor

## 1.1 System overview

Native Rust development profile for local Java source `JavaSupportRuntime/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

## 1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

## 1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

## 1.4 Implemented behavior

Revision subscriptions, strict diagnostic intake, atomic history/outbox, queries

## 1.5 Open release gates

AMQP/MQTT workers and orchestration reconciliation absent; status degraded; no automated retention; history cap100000. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

## 1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `echo`, `generalManagement`, `sysMonitorHistory`, `sysMonitorManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

## 1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

| Method | Path | Access / status |
|---|---|---|
| GET | /sysmonitor/general/mgmt/get-config | management |
| POST | /sysmonitor/general/mgmt/logs | management |
| GET | /sysmonitor/monitor/echo | certificate |
| GET | /sysmonitor/management/status | management |
| GET | /sysmonitor/management/providers | management |
| GET | /sysmonitor/management/subscriptions | management |
| PUT | /sysmonitor/management/subscriptions/{providerSystem} | management |
| DELETE | /sysmonitor/management/subscriptions/{providerSystem} | management |
| POST | /sysmonitor/management/reconcile | management / 501 |
| GET | /sysmonitor/history/systems | identity |
| GET | /sysmonitor/history/systems/{systemName}/latest | identity |
| GET | /sysmonitor/history/systems/{systemName}/observations | identity |
| POST | /sysmonitor/observations | management |

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

## 2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.